

Employee Registration & Login System

Rough Draft Planning Document

Programming Assessment 1 | Lubanzi ICT Academy

Date: 22/02/2026

1. Objective

The goal of this assessment is to build a simple C# console application that allows employees to register new accounts and log in. The program must demonstrate a solid understanding of data types, access modifiers, if statements, and in-memory list storage.

This document outlines my step-by-step plan for approaching the assignment before writing any code.

2. Understanding the Requirements

Before writing any code, I read through the full assessment brief and identified the three core features the program must have:

- Register New Employee — collect all employee details and store them in a list
- Login — verify a username and password, then display the employee's profile
- View All Employees — loop through the list and print each employee

I also noted the technical concepts I need to demonstrate in the code:

Concept	How I will use it
Data Types	string, int, decimal, bool for the Employee properties
Access Modifiers	public for properties, private for password field, internal for the class
if Statements	For all validation: age, empty input, username uniqueness, login checks
ArrayList<Employee>	Store registered employees in memory — no files or database

3. Class Design Plan

I will create two classes: Employee and Program. The Employee class holds all the data and a method to verify passwords. The Program class runs the menu and calls the correct methods.

3.1 Employee Class

The Employee class will be marked internal (accessible only within this project). It will have the following properties:

Property	Type & Access
Name	public string
Surname	public string
Age	public int
Salary	public decimal
Role	public string
Username	public string
Password	private string (field, not property)
EmployeeID	public int — private setter
IsActive	public bool

I will also add a public CheckPassword(string input) method that returns true or false. This keeps the password field private — no outside code can read the password, but the Program class can still verify a login attempt.

3.2 Program Class

The Program class will contain:

- A List<Employee> to store all registered employees in memory
- A nextID counter starting at 1001 for auto-incrementing employee IDs(Similar MS Access DataBase)
- A Main() method with a loop showing the main menu
- A RegisterEmployee() method
- A LoginEmployee() method
- A ViewAllEmployees() method
- Helper methods: ReadNonEmpty() and UsernameExists()

4. Logic Plan (Before Coding)

This section describes in plain English what each part of the program will do.

4.1 Main Menu Loop

The program will run inside a while loop. It will show the menu and read the user's choice. Based on the choice it will call the correct method, or exit the loop if the user picks 4.

```
WHILE running is true:
    Show menu (Register / Login / View All / Exit)
    Read user choice
    IF choice == 1 → call RegisterEmployee()
    IF choice == 2 → call LoginEmployee()
    IF choice == 3 → call ViewAllEmployees()
    IF choice == 4 → set running = false
    ELSE → show invalid option message
```

4.2 Registration Logic

The register method will collect each field one at a time, validating as it goes. I plan to tackle the validations in this order:

1. Collect name and surname — check they are not empty
2. Collect age — check it is a valid integer and is 18 or older
3. Collect salary — check it is a valid decimal and not negative
4. Collect role — check it is not empty
5. Collect username — check it is not empty and does not already exist in the list
6. Collect password — check it is not empty
7. Create a new Employee object with the next available ID
8. Add the employee to the list and show a success message with the ID

For the username check, I will loop through the employees list and compare each username to the one entered. If a match is found, I will ask the user to try a different username.

4.3 Login Logic

The login method will:

9. Ask for username and password
10. Search the list for an employee with that username
11. If not found: show 'Username not found' error and return
12. If found: call CheckPassword() on that employee
13. If password is wrong: show 'Incorrect password' error and return
14. If password is correct: show welcome message and call DisplayProfile()

4.4 View All Employees

This method will check if the list is empty first. If it is, it will show a message saying no employees are registered yet. Otherwise it will loop through the list and print the ID, name, role, and active status for each employee.

5. if Statements I Will Use

The assessment requires me to demonstrate if statements for the following cases. I have planned each one, it is used to verify if the employee registered is too young or not:

Validation	My planned if statement
Age must be 18+	if (age < 18) → show error, loop back
Empty input	if (string.IsNullOrEmpty(input)) → show error, loop back
Username taken	if (UsernameExists(username)) → show error, ask again
Salary invalid	if (!decimal.TryParse(...) salary < 0) → show error
User not found	if (found == null) → show error, return
Wrong password	if (!found.CheckPassword(password)) → show error, return
Empty list	if (employees.Count == 0) → show 'no employees' message

6. Data Types Plan

I chose each data type deliberately based on what the data represents:

- string — for Name, Surname, Role, Username, and Password because these are text values.
- int — for Age and EmployeeID because both are whole numbers. An age of 28.5 makes no sense.
- decimal — for Salary because decimal is more accurate with money than double. It avoids floating-point rounding errors.
- bool — for IsActive to represent a simple true/false state (active or inactive account).

7. Access Modifiers Plan

I am planning my access modifiers to follow good encapsulation practice:

- internal — for the Employee class itself, so it is only usable within this project.

- public — for all properties that the Program class needs to read (Name, Surname, Age, Salary, Role, Username, EmployeeID, IsActive).
- private — for the _password field because no outside code should ever read or change a password directly.
- private setter on EmployeeID — the ID can be read from outside but can only be set inside the constructor.
- public methods — CheckPassword() and DisplayProfile() need to be callable from Program.

8. Test Cases I Plan to Run

After writing the code I will test the following scenarios to confirm everything works:

Test	Expected result
Register a new employee with valid data	Success message + Employee ID shown
Register with same username	Error: username already taken
Register with age 17	Error: must be 18 or older
Register with an empty name field	Error: field cannot be empty
Login with correct credentials	Welcome message + profile displayed
Login with wrong password	Error: incorrect password
Login with non-existent username	Error: username not found
View all employees (none registered)	Message: no employees yet
View all employees (some registered)	List of all employees printed

9. Planned Development Order

I plan to build the program in this order to keep things manageable:

15. Write the Employee class with all properties, the constructor, CheckPassword(), and DisplayProfile()
16. Write the main menu loop in Program with just the four options printing to the console

17. Build RegisterEmployee() step by step, adding one validation at a time
18. Build LoginEmployee() — username search first, then password check
19. Build ViewAllEmployees() — empty check, then the loop
20. Add the ReadNonEmpty() and UsernameExists() helper methods
21. Run all test cases from Section 8 and fix any issues
22. Write the final explanation document as required by Section 3.6 of the brief

10. Potential Challenges

I have identified a few areas that may need extra care:

- Keeping the password secure — I need to make sure the `_password` field is truly private and that I only ever interact with it through the `CheckPassword()` method.
- Input validation loops — getting the while loops right so the program keeps asking until valid input is given, without crashing on bad input like letters entered for age.
- Salary parsing — `decimal.TryParse()` needs to be used carefully so the program does not crash if the user types something like 'abc' as a salary.
- Username case sensitivity — I plan to use `.ToLower()` on both sides of the comparison so that 'JohnDoe' and 'johndoe' are treated as the same username.
- Linking the Registration and Login data for verification will be challenging, especially since they'll be store on temporary memory.

End of my Planning Document